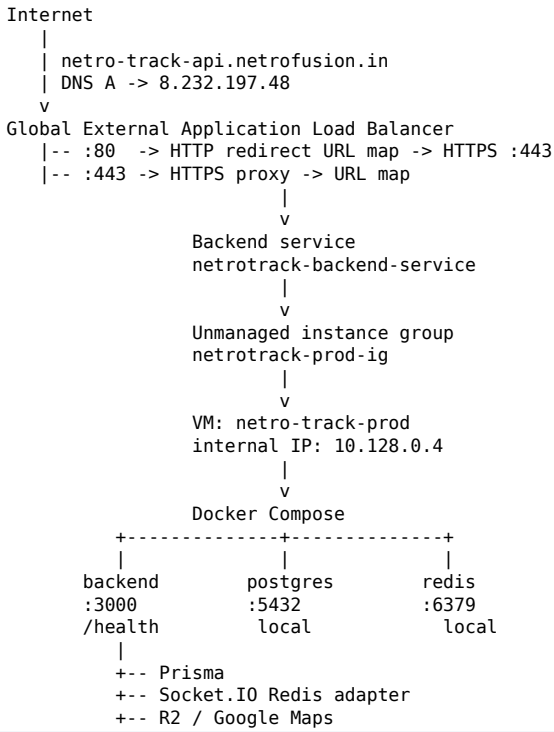
NetroTrack Production — Complete GCP Deployment Runbook

End-to-end VM, Docker Compose, Load Balancer, DNS, TLS and redirect implementation

1. Executive summary

This document records the production flow actually implemented and verified for NetroTrack: a single GCP VM running Docker Compose (backend, PostgreSQL and Redis), exposed through a Google Cloud external Application Load Balancer, with a reserved global IP, Google-managed TLS certificate, DNS at GoDaddy, and HTTP→HTTPS redirect.

2. Final verified architecture



Important: PostgreSQL and Redis were intentionally bound to 127.0.0.1 on the VM. Backend port 3000 is the only application port reached by the load balancer.

3. Environment and resource identifiers

```
GCP project:          netro-track-prod
VM:                    netro-track-prod
Zone:                  us-central1-a
VPC:                   default
Subnet:                default (us-central1)
VM internal IP:        10.128.0.4
Reserved global IP:    8.232.197.48
DNS hostname:          netro-track-api.netrofusion.in

Backend container:     netrotrack-backend
Postgres container:    netrotrack-postgres
Redis container:       netrotrack-redis
Instance group:        netrotrack-prod-ig
Backend service:       netrotrack-backend-service
Health check:          netrotrack-backend-health
HTTPS certificate:     netrotrack-api-cert
HTTPS proxy:           netrotrack-prod-https-proxy
HTTP redirect map:     netrotrack-http-redirect-map
HTTP redirect proxy:   netrotrack-prod-http-redirect-proxy
HTTP forwarding rule:  netrotrack-prod-http-rule
```

```
HTTPS forwarding rule:  netrotrack-prod-https-rule
```

4. Docker Compose — production configuration

This is the Compose configuration used during the verified deployment. Secrets remain in the VM .env file and are represented here as variables.

```
services:
  postgres:
    image: postgres:16
    container_name: netrotrack-postgres
    restart: unless-stopped
    environment:
      POSTGRES_DB: ${POSTGRES_DB}
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
    volumes:
      - ./postgres/data:/var/lib/postgresql/data
    ports:
      - "127.0.0.1:5432:5432"
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER} -d ${POSTGRES_DB}"]
      interval: 10s
      timeout: 5s
      retries: 5

  redis:
    image: redis:7-alpine
    container_name: netrotrack-redis
    restart: unless-stopped
    command:
      - redis-server
      - --requirepass
      - ${REDIS_PASSWORD}
      - --maxmemory
      - 768mb
      - --maxmemory-policy
      - allkeys-lru
    ports:
      - "127.0.0.1:6379:6379"
    healthcheck:
      test: ["CMD", "redis-cli", "-a", "${REDIS_PASSWORD}", "ping"]
      interval: 10s
      timeout: 5s
      retries: 5

  backend:
    image: ghcr.io/netrofusionlabs/netro_track-backend:production
    container_name: netrotrack-backend
    restart: unless-stopped
    depends_on:
      postgres:
        condition: service_healthy
      redis:
        condition: service_healthy
    environment:
      NODE_ENV: production
      PORT: 3000
      DATABASE_URL: postgresql://${POSTGRES_USER}:${POSTGRES_PASSWORD}@postgres:5432/${POSTGRES_DB}?schema=public
      REDIS_URL: redis://:${REDIS_PASSWORD}@redis:6379
      JWT_ACCESS_SECRET: ${JWT_ACCESS_SECRET}
      JWT_REFRESH_SECRET: ${JWT_REFRESH_SECRET}
      LOG_LEVEL: ${LOG_LEVEL:-info}
      R2_ACCOUNT_ID: ${R2_ACCOUNT_ID}
      R2_ACCESS_KEY_ID: ${R2_ACCESS_KEY_ID}
      R2_SECRET_ACCESS_KEY: ${R2_SECRET_ACCESS_KEY}
      R2_BUCKET_NAME: ${R2_BUCKET_NAME}
      R2_PUBLIC_URL: ${R2_PUBLIC_URL}
      GOOGLE_MAPS_API_KEY: ${GOOGLE_MAPS_API_KEY}
    expose:
      - "3000"
```

5. Application health verification

```
# On the VM
cd /opt/netrotrack
docker compose ps
curl -i http://127.0.0.1:3000/health

# If curl/wget is not installed inside the slim backend image:
docker compose exec backend node -e "require('http').get('http://127.0.0.1:3000/health', r => { console.log('HTTP', r.statusCode); r

# Inspect compiled route
```

```
docker compose exec backend sh -c 'grep -R "health" /app/apps/backend/dist -n | head -20'
```

Final verified response: HTTP 200 and {"success":true,"message":"Server is healthy"}. The root path / returned 404 with “Cannot GET /”; this was not an application failure because /health is the intended health endpoint.

6. Load balancer backend

```
# VM tag
gcloud compute instances add-tags netro-track-prod   --project=netro-track-prod   --zone=us-central1-a   --tags=netrotrack-backend

# Firewall
gcloud compute firewall-rules create netrotrack-allow-lb   --project=netro-track-prod   --network=default   --direction=INGRESS   --

# Unmanaged instance group
gcloud compute instance-groups unmanaged create netrotrack-prod-ig   --project=netro-track-prod   --zone=us-central1-a

gcloud compute instance-groups unmanaged add-instances netrotrack-prod-ig   --project=netro-track-prod   --zone=us-central1-a   --in

gcloud compute instance-groups unmanaged set-named-ports netrotrack-prod-ig   --project=netro-track-prod   --zone=us-central1-a   --

# Health check
gcloud compute health-checks create http netrotrack-backend-health   --project=netro-track-prod   --port=3000   --request-path=/heal
```

7. Verified backend service state

```
gcloud compute backend-services describe netrotrack-backend-service   --project=netro-track-prod   --global   --format="yaml(name,pr

gcloud compute instance-groups unmanaged list-instances netrotrack-prod-ig   --project=netro-track-prod   --zone=us-central1-a

gcloud compute backend-services get-health netrotrack-backend-service   --project=netro-track-prod   --global   --format="yaml"
```

Final observed state: instance 10.128.0.4 on port 3000 was HEALTHY. A duplicate-backend error occurred when attempting to add the same instance group a second time; inspection showed it was already attached, so no second add was required.

8. DNS and static IP

```
gcloud compute addresses describe netrotrack-prod-ip   --project=netro-track-prod   --global   --format="get(address)"

dig +short netro-track-api.netrofusion.in
```

Verified: the DNS A record resolves to 8.232.197.48.

9. HTTPS certificate and proxy

```
gcloud compute ssl-certificates create netrotrack-api-cert   --project=netro-track-prod   --global   --domains=netro-track-api.netro

gcloud compute target-https-proxies create netrotrack-prod-https-proxy   --project=netro-track-prod   --url-map=netrotrack-prod-url-

gcloud compute forwarding-rules create netrotrack-prod-https-rule   --project=netro-track-prod   --global   --target-https-proxy=net

gcloud compute ssl-certificates describe netrotrack-api-cert   --project=netro-track-prod   --global   --format="yaml(name,type,mana
```

The certificate moved from PROVISIONING to ACTIVE. TLS was independently verified with openssl and curl.

10. HTTPS verification

```
openssl s_client   -connect netro-track-api.netrofusion.in:443   -servername netro-track-api.netrofusion.in

curl -I https://netro-track-api.netrofusion.in/health
curl -IL http://netro-track-api.netrofusion.in/health
```

Final verified result: HTTPS returns HTTP/2 200, certificate verification is OK, and HTTP returns 301 followed by HTTPS 200.

11. HTTP → HTTPS redirect YAML

```
name: netrotrack-http-redirect-map

defaultUrlRedirect:
  httpsRedirect: true
  stripQuery: false

cat > /tmp/netrotrack-http-redirect.yaml <<'EOF'
```

```
name: netrotrack-http-redirect-map

defaultUrlRedirect:
  httpsRedirect: true
  stripQuery: false
EOF

gcloud compute url-maps import netrotrack-http-redirect-map   --project=netro-track-prod   --source=/tmp/netrotrack-http-redirect.ya
```

12. HTTP redirect proxy and forwarding rule

```
gcloud compute target-http-proxies create netrotrack-prod-http-redirect-proxy   --project=netro-track-prod   --url-map=netrotrack-ht

gcloud compute forwarding-rules create netrotrack-prod-http-rule   --project=netro-track-prod   --global   --target-http-proxy=netro

gcloud compute forwarding-rules list   --project=netro-track-prod   --global   --format="table(name,IPAddress,IPProtocol,portRange,t
```

13. Final production validation checklist

Check	Expected / verified
VM	RUNNING; internal IP 10.128.0.4
PostgreSQL	Up / healthy; bound to 127.0.0.1:5432
Redis	Up / healthy; bound to 127.0.0.1:6379
Backend	Up; port 3000
App health	HTTP 200 /health
Instance group	netrotrack-prod-ig; size 1
LB health	HEALTHY on 10.128.0.4:3000
DNS	netro-track-api.netrofusion.in → 8.232.197.48
Certificate	ACTIVE
HTTPS	HTTP/2 200 /health
HTTP	301 → HTTPS, then 200